

JSON

- JSON stands for **J**ava**S**cript **O**bject **N**otation.
- JSON is a lightweight text format for storing and transporting data.
- JSON is often used when data is sent from a server to a web page.
- JSON is "self-describing" and easy to understand.
- A JSON object is an unordered set of name/value pairs.

JSON - DATATYPES

In JSON, values must be one of the following data types:

- a string
- a number
- an object (JSON object)
- an array
- a boolean
- *null*

JSON values **cannot** be one of the following data types:

- a function
- a date
- *undefined*

JSON Object:

```
var a= {  
  "name": "Test",      // String  
  "age": 25,           // Number  
  "ispass": false,    // Boolean  
  "address": {         // Object (Nested JSON)  
    "city": "Ahmedabad",  
    "zip": "380015"  
  },  
  "subjects": ["Math", "Science"], // Array  
  "year": null        // Null  
}  
console.log(a);  
console.log(a.age)  
console.log(a['address'])  
console.log(a['address']['city'])
```

Output:

```
{
  name: 'Test',
  age: 25,
  ispass: false,
  address: { city: 'Ahmedabad', zip: '380015' },
  subjects: [ 'Math', 'Science' ],
  year: null
}
25
{ city: 'Ahmedabad', zip: '380015' }
Ahmedabad
```

Example of an array of objects

```
var cars = [{color: 'gray', model: '1', nOfDoors: 4},
            { color: 'yellow',model: '2', nOfDoors: 4}];
```

Methods to Access Data in JSON / Objects

Method	Explanation	Syntax	
Dot Notation	Access a value directly using the key with a dot (.). It is the most common and simplest way when the key name is known.	<code>object.key</code>	
Bracket Notation	Access a value using square brackets. This method is useful when the key name is stored in a variable or when the key contains spaces or special characters.	<code>object["key"]</code>	
Nested Object Access	Used when one object exists inside another object. Multiple dots are used to reach the inner value.	<code>object.key.key</code>	
Array Inside Object	Used when an array is stored inside an object. Access the array using the key and then use the index number to get a specific element.	<code>object.array[index]</code>	

JavaScript built in functions

1. Converting JSON strings into JavaScript objects: **JSON.parse()**
2. Converting an object into a JSON string: **JSON.stringify()**

JSON.parse()

Example:

```
<script>
let obj = JSON.parse('{"var1":"LJ","var2":"University"}');
console.log(obj);
console.log(obj.var1 + " " + obj.var2);
</script>
```

Output:

```
{var1: 'LJ', var2: 'University'}
LJ University
```

JSON.stringify()

Example:

```
<script>
let obj = JSON.stringify({var1: 'LJ', var2: 'University'});
console.log(obj);
</script>
```

Output:

```
'{"var1":"LJ","var2":"University"}'
```

Updating Values of Existing Object Keys

```
let person = {name:"abc",age: 17, city:"Ahmedabad"};
```

```
person.age = 30;           // update value of age using Dot Notation
console.log(person);        //change in person value
```

```
person["name"] = "xyz"    //update value of name using Bracket Notation
console.log(person);        //change in age value
```

```
let key = "city";          // Update value Using Variable Key
person[key] = "Gandhinagar";
console.log(person);        // change in city value
```

Output:

```
{ name: 'abc', age: 30, city: 'Ahmedabad' }
{ name: 'xyz', age: 30, city: 'Ahmedabad' }
{ name: 'xyz', age: 30, city: 'Gandhinagar' }
```

Method to create JSON object

Syntax: obj[key]=value

eg.

```
var obj2={}  
obj2["a"]=111  
console.log(obj2)
```

output:

```
{“a”:111}
```

Conversion of Array to object

```
var data = ["a","b","c"];  
function fun(n) {  
  var arr = {};  
  arr[n[0]] = n[n.length - 1];  
  //arr[1] = n[1] ----- gives {1: 'b'}  
  return arr;  
}  
console.log(fun(data));  
Output: { a: 'c' }
```

Node JS-1

Download node js from <https://nodejs.org/en> and install it. To check downloaded version type `node -v` in console.

The Node.js installer includes the NPM(Node Package Manager). For version check `npm -v`.

Node.js is an open source server environment.

Node.js allows you to run JavaScript on the server.

Callback

A callback function is a function passed as an argument to another function and executed after the main function finishes its task.

JavaScript setTimeout() Method: This method executes a function, after waiting a specified number of milliseconds.

```
const message = function() {  
    console.log("This message is shown after 3 seconds");  
}  
setTimeout(message, 3000);
```

JavaScript setInterval() Method: The `setInterval()` method repeats a given function at every given time interval.

Display clock using setInterval Method

```
function updateTime() {  
    // Get the current time in HH:MM:SS format for India timezone  
    const timeString = new Date().toLocaleTimeString()  
    console.log(timeString) }  
updateTime();  
  
// Call updateTime every second (1000 milliseconds)  
setInterval(updateTime, 1000);
```

File System Module

The Node.js file system module allows you to work with the file system on your computer. To include the File System module, use the `require()` method:

```
var fs = require('fs');
```

Synchronous mode

	Syntax	example
Create folder	<code>fs.mkdirSync(folder_name)</code>	<code>fs.mkdirSync("Details")</code>
Create file	<code>fs.writeFileSync(file_name,data)</code>	<code>fs.writeFileSync("user.txt","Hello")</code>
append the data	<code>fs.appendFileSync(file_name,data)</code>	<code>fs.appendFileSync("user.txt","Hi")</code>
Read data	<code>fs.readFileSync(filr_name)</code> note: it gives buffer data so convert it in string using “ utf-8 ” or .toString() method.	<code>var data=fs.readFileSync("user.txt","utf-8")</code> console.log(data) or fs.writeFilesync(JSON.stringify(data))
Rename file	<code>fs.renameSync(file_name, file_new_name)</code>	<code>fs.renameSync("user.txt","user1.txt")</code>
Copy File	<code>fs.copyFileSync(source_file, destination_file)</code>	<code>fs.copyFileSync("user1.txt", "user2.txt")</code>
Delete file	<code>fs.unlinkSync(file_name)</code>	<code>fs.unlinkSync("user.txt")</code>
Delete Folder	<code>fs.rmdirSync(folder_name)</code>	<code>fs.rmdirSync("Details")</code>

Asynchronous mode

	Syntax	example
Create folder	fs.mkdir(folder_name,callback)	fs.mkdir("details", (err)=> { if (err) throw err})
Create file	fs.writeFile(file_name,data,callback)	fs.writeFile('test.txt', 'Hello World!', (err)=> {..})
append the data	fs.appendFile(file_name,data,callback)	fs.writeFile('test.txt', 'Hi', (err)=> {..})
Read data	fs.readFile(file_name,encoding(optional), callback) Note: It gives buffer data so convert it in string using "utf-8" or .toString() method.	fs.readFile('test.txt', 'utf-8', (err,data)=>{ if (err){ console.log(err) } else { console.log(data) } })
Delete file	fs.unlink(file_name,callback)	fs.unlink('test.txt', (err)=> {..})
Delete Folder	fs.rmdir(folder_name,callback)	fs.rmdir("details", (err)=> {..})
Rename File	fs.rename(file_name, file_new_name, callback)	fs.rename('old.txt', 'new.txt', (err)=> {..})
Copy File	fs.copyFile(source_file,destination_file ,callback)	fs.copyFile('source.txt', 'destination.txt', (err)=> {..})

Async Flow

```

fs.writeFile("a.txt", "Hello", (err) => {
    fs.appendFile("a.txt", " World", (err) => {
        fs.readFile("a.txt", "utf-8", (err, data) => {
            console.log(data);
            .....
        });
    });
});

```

Path Module

Mehod	Description
basename()	Returns the last part of a path (file name)
dirname()	Returns the directories of a path
extname()	Returns the file extension of a path
parse()	Returns an object containing path information

Example:

```
var pm=require("path");
path1=pm.dirname("D:/FSD-2/node/addon.txt");
console.log("Path: " + path1);
path2=pm.extname("D:/FSD-2/node/addon.txt");
console.log("Extension: "+path2);
path2=pm.basename("D:/FSD-2/node/addon.txt");
console.log("Basename: "+ path2);
path2=pm.parse("D:/FSD-2/node/addon.txt");
console.log(path2);
console.log(path2.root);
console.log(path2.dir);
console.log(path2.base);
console.log(path2.ext);
console.log(path2.name);
```

Output:

```
Path: D:/FSD-2/node
Extension: .txt
Basename: addon.txt
{
  root: 'D:/',
  dir: 'D:/FSD-2/node',
  base: 'addon.txt',
  ext: '.txt',
  name: 'addon'
}
D:/
D:/FSD-2/node
addon.txt
.txt
addon
```


Events

Method	Syntax	Description
.on() / .addListener()	ee.on(event, listener)	Bind function with event.
emit()	ee.emit(event)	Trigger an event.
removeListener()	ee.removeListener(event, listener)	Removes a specific listener from event.
removeAllListeners()	ee.removeAllListeners(event)	Removes all listeners from event.
listenerCount()	ee.listenerCount(event)	Returns number of listeners associated with event.

Steps for event handling script

```
// 1) Import "events" module
var e=require("events");
// 2) Create EventEmitter object
var ee=new e();
// 3) Bind connection event with the handler/function
ee.on("sayName",()=>{
    console.log("your name is xyz")
});
// 4) Emit / Fire connection event
ee.emit("sayName");
```

Example: Registering for the event with call-back parameter.

```
var EventEmitter =require("events");
var ee=new EventEmitter();
ee.on("sayName",(statusCode,msg)=>{
    console.log(`status code id ${statusCode} and page is ${msg}`);
});
ee.emit("sayName",200,"ok");
```

Output:

status code id 200 and page is ok

Example: Write a NodeJs script to create two listeners for a common event. Call their respective callbacks. Print no. of events associated with an emitter. Remove one of the listener and print no of remaining listeners.

```
var event = require('events');
var ee = new event();

const listener1 = () => {
  console.log("listener1 executed");
};

const listener2 = () => {
  console.log("listener2 executed");
};

ee.addListener("conn", listener1);
ee.on("conn", listener2);

let count = ee.listenerCount("conn"); // counts listeners for conn event
console.log(`${count} for conn event`);

ee.emit("conn");

ee.removeListener("conn", listener1); // remove listener1

count = ee.listenerCount("conn");
console.log(`${count} for conn event`);

ee.emit("conn");

// Remove all listeners
ee.removeAllListeners("conn");
count = ee.listenerCount("conn");
console.log(`Again Count after removing all listeners: ${count}`);
```

Output:

2 for conn event

listener1 executed

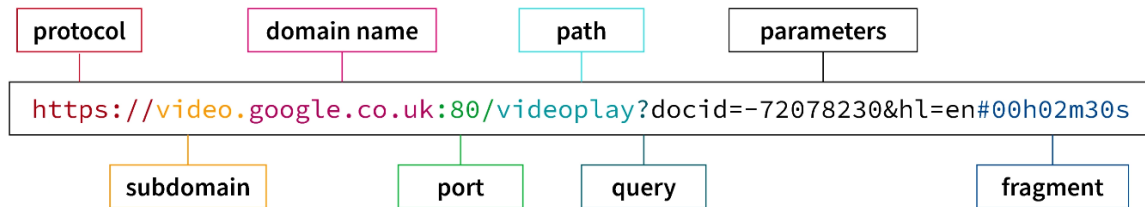
listener2 executed

1 for conn event

listener2 executed

Again Count after removing all listeners: 0

URL module



```
var url=require("url");
```

url.parse() — This method takes the url string as a parameter and parses it. The url module returns an object with each part of the url as property of the object.

Syntax:

```
url.parse(url_string, parse_query_string, slashes_host)
```

The returned object contains the following properties

- **protocol**: This specifies the protocol scheme of the URL (ex: https:).
- **auth**: This specifies the authentication segment of the URL (ex: username: password).
- **host**: This specifies the host name with the port number.
- **hostname**: This specifies the host name excluding the port number.
- **port**: Indicates the port number.
- **pathname**: Indicates the URL path.
- **query**: This contains a parsed query string unless parsing is set to false.
- **hash**: It specifies the fragment portion of the URL **including the “#”**.
- **href**: It specifies the complete URL.
- **origin**: It specifies the origin of the URL.
- **etc...**

Example

```
var u=require("url");
var adr1="https://www.google.com/search?q=good+morning";
var q=u.parse(adr1,true); //query will be given as JSON Object
console.log(q);
```

Output:

```

Url {
  protocol: 'https:',
  slashes: true,
  auth: null,
  host: 'www.google.com',
  port: null,
  hostname: 'www.google.com',
  hash: null,
  search: '?q=good+morning',
  query: [Object: null prototype] { q: 'good morning' },
  pathname: '/search',
  path: '/search?q=good+morning',
  href: 'https://www.google.com/search?q=good+morning'
}

```

Impact of parse_query_string value

Default = false -> return values of query in string

Set true -> returns values of query in object

For example

```

var u=require("url");
var data = "http://localhost:8080/info?age=20";
var q1=u.parse(data);
console.log(q1.query);

```

Property	url.parse(data)	url.parse(data, true)
query	'age=20'	[Object: null prototype] {age: '20'}
Type	String	Object
Access age	Not direct so undefined	q.query.age → 20

Note:

Dot notation only works with **valid identifier names** (no special characters like ; or &) **For rest always use bracket notation.**

Example:

```

const link = https://www.google.com/exam.txt?a1=Hello&a2=HowRU?
const addr = url.parse(link,true);
const a2 = addr.query['&a2'];
console.log(a2)

```

HTTP Module: Render Response, Read HTML File Server, Routing

Example to create server and print “Hello world” message in **index.js** file

```
var http = require('http');
//create a server object
http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.write('<h1>Hello World!</h1>'); //write a response to the client
  res.end(); //end the response can be empty or include string
}).listen(8080); //the server object listens on port 8080
```

Run file by **node index.js** in terminal and hit <http://localhost:8080> on browser

Output on browser → <http://localhost:8080/>

Hello World!

Note: Remember Status code: 200 – ok/ 404 :Page not found

Types of HTTP Header {"content-type":"MIME type"}

Name	MIME type
HyperText Markup Language (HTML)	text/html
Cascading Style Sheets (CSS)	text/css
JavaScript	application/javascript
JavaScript Object Notation (JSON)	application/json
JPEG Image	image/jpeg
Portable Network Graphics (PNG)	image/png

Create HTTP webpage on which home page display “Home page”, student page shows “Student page” and any other page shows “Page Not found”.

(Render Response & Routing)

```
var h=require("http");
var server = h.createServer((req,res)=>{
  if(req.url=="/"){
    res.writeHead(200,{"content-type":"text/html"});
    res.write("<b> Home page </b>");
    res.end();
  }
  else if(req.url=="/student"){
```

```
res.writeHead(200,{ "content-type":"text/plain"}); //plain shows code as it is
res.write("<i> Home page1 </i>");
res.end();
}
else {
  res.writeHead(404,{ "content-type":"text/html"});
  res.write("<h1> Page Not found </h1>");
  res.end("Thanks");
} });
server.listen(5001);
console.log("Thanks for run");
```

JSON Example

```
var http = require('http');
http.createServer((req, res) =>{
  // Tell browser this is JSON
  res.writeHead(200, { 'Content-Type': 'application/json' });
  const data = {
    Subject: "FSD-2",
    Topic: "Node.js"
  };
  res.end(JSON.stringify(data));
}).listen(8180);
```

Serving Static Image with Node.js HTTP Module

```
var http = require('http');
var fs = require('fs');

http.createServer((req, res) =>{

  if (req.url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>Hello</h1>');
  }

  else if (req.url === '/logo192.png') {
    const img = fs.readFileSync('logo192.png');
    res.writeHead(200, { 'Content-Type': 'image/png' });
    res.end(img);
  }

}).listen(8180);
```

Note: Each resource like HTML, image, CSS, or JavaScript is a separate HTTP request and must be handled by the server.

Extra Examples

Task -1

Write code to perform the tasks as asked below.

- Add three buttons.
- Increase button to increase the fonts. It should stop increasing the fonts when the font size reaches to 200px or stop button is clicked.
- Decrease button to decrease the fonts. It should stop decreasing the fonts when the font size reaches to 20px or stop button is clicked.
- Stop button to stop increasing or decreasing the fonts.
- Increasing/decreasing interval is of 100 ms.
- (Default font size = 50px)

```
<html>
<body>
<p id="p1" style="font-size: 50px;"> Hello</p>
<button onclick="inc()">Increase</button>
<button onclick="dec()">Decrease</button>
<button onclick="stop()">stop</button>
<script>
font=50;
function fun(font){
document.getElementById('p1').style.fontSize=font+'px';
}
function inc()
{
test = setInterval(()=>{
if(font<100){ fun(++font);
}
},100);
}
function dec()
{
test =setInterval(()=>{
if(font>15){ fun(--font);}
},100);
}
function stop() { clearInterval(test); }
</script>
</body>
</html>
```

Task -2

Write a node js Asynchronous program to perform CRUD operation of file management which should perform below task in sequence. **[Callback Hell]**

1. Create folder named "hello"
2. create a file in it named a.txt and add some data in it.
- 3.add more data at last in file
4. read data without getting buffer at first
5. rename the file
6. delete both the file and folder.

```
const fs = require('fs');
const additionalData = '\nMore data added at last.';

fs.mkdir("hello", (err) => {
    if (err) throw err;
    console.log('Folder created successfully.');
```

```
    fs.writeFile("hello/a.txt", 'Some initial data.', (err) => {
        if (err) throw err;
        console.log('File created successfully.');
```

```
        fs.appendFile("hello/a.txt", additionalData, (err) => {
            if (err) throw err;
            console.log('Data appended successfully.');
```

```
            fs.readFile("hello/a.txt", 'utf8', (err, data) => {
                if (err) throw err;
                console.log('File content:', data);
```

```
                fs.rename("hello/a.txt", "hello/a1.txt", (err) => {
                    if (err) throw err;
                    console.log('File renamed successfully.');
```

```
                    fs.unlink("hello/a1.txt", (err) => {
                        if (err) throw err;
                        console.log('File deleted successfully.');
```

```
                        fs.rmdir("hello", (err) => {
                            if (err) throw err;
                            console.log('Folder deleted successfully.');
```

```
                        });
                    });
                });
            });
        });
    });
}
```


Task-3

Write a node js script to write the text “This is data” to new.txt file. After that append the text “that is data” to same ne .txt file. After that read the file and print file concept on console. After finishing read operation, print the line “Thanks for using my program” on console. All read/write operations are asynchronous. (Using Event)

```
const fs = require('fs');
const e = require('events');
const ee = new e();
ee.on('write', () => {
  fs.writeFile("new.txt", 'this is data', () => {
    console.log('Successfully wrote to file.');
```

```
    ee.emit('append'); // trigger event after write finishes
  });
});
ee.on('append', () => {
  fs.appendFile("new.txt", '\nthat is data', () => {
    console.log('Successfully appended to file.');
```

```
    ee.emit('read');
  });
});
ee.on('read', () => {
  fs.readFile("new.txt", 'utf8', (err, data) => {
    if (err) throw err;
    console.log(data);
    ee.emit('finish');
  });
});
ee.on('finish', () => {
  console.log('Thanks for using my program.');
```

```
});
ee.emit('write'); // Process Start
```

Task-4

Create a Node.js program using EventEmitter where the first event calls a second event.

```
const EventEmitter = require('events');
const event = new EventEmitter();
event.on("first", () => { // First event
  console.log("First event executed");
  // Trigger second event
  event.emit("second");
});
event.on("second", () => { // Second event
  console.log("Second event executed");
});
event.emit("first"); // Start the chain
```